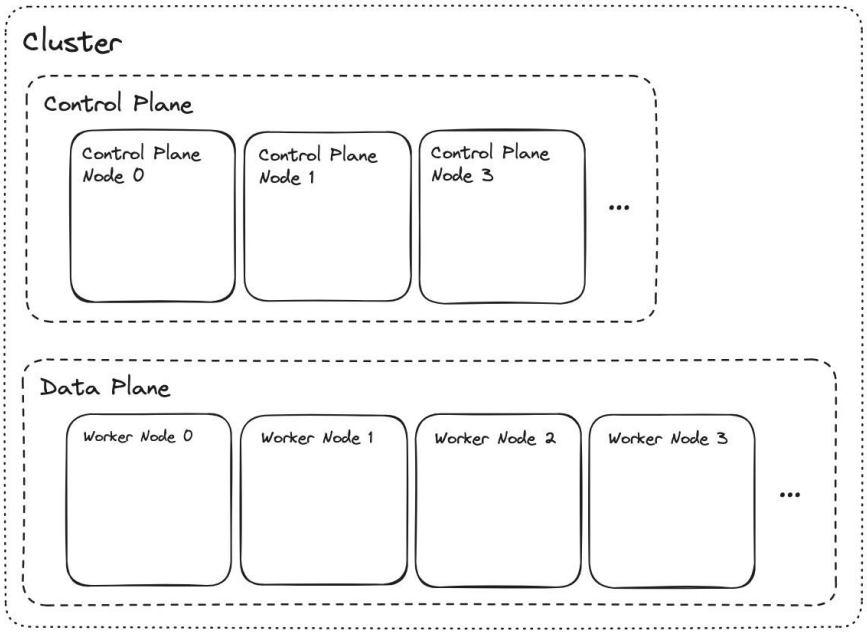
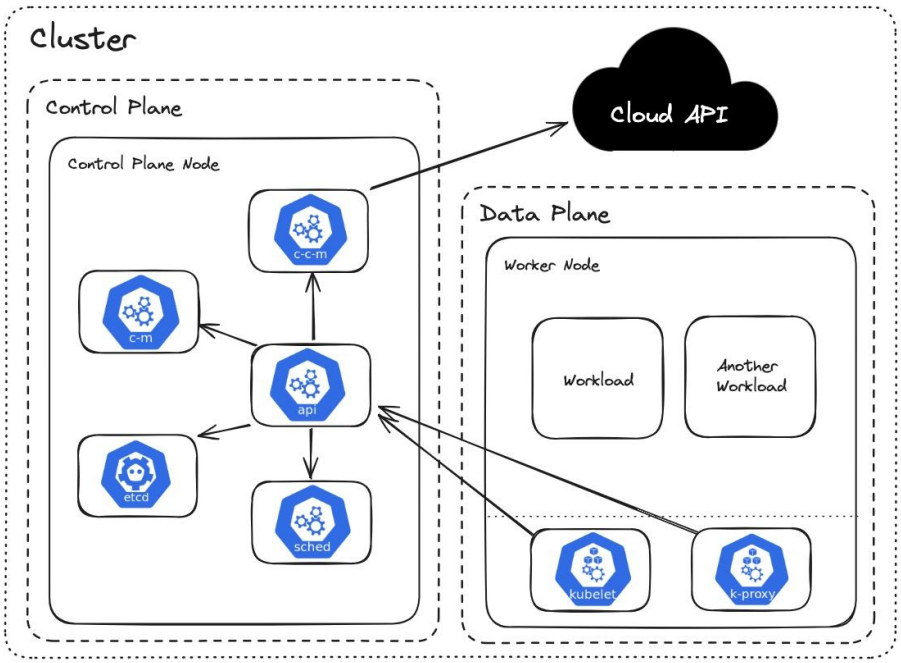
- GitHub: https://github.com/sidpalas/devops-directive-kubernetes-course/tree/main
- **Planes and Nodes**



  - Nodes are either bare metal or virtual machines that is running the applications.
  - Nodes are divided in 2 parts, one is ***Control Plane*** and another is ***Data Plane***.
  - The planes combined forms ***Cluster***.
  - Control Plane (Brain of Kubernetes)
    - It is where the system components run.
    - It makes decisions, manages cluster, schedules the workloads.
    - It decides where pods should run, keeps track of cluster state, handles API requests.
    - It includes components like API Server, Scheduler, Controller Manager, etcd (database).
  - Data Plane (Workers)
    - It is where the user-end applications run.
    - It actually runs your applications (pods/containers)
    - Each worker node runs Pods (your app), Kubelet (talks to control plane), Container runtime (Docker/containerd)
  - The flow will be like below:
    - You deploy an app (**kubectl apply**)
    - Control plane decides which worker node should run it.
    - Worker node runs the container (your app)
    - Control plane keeps checking if everything is healthy.

- Analogy:
  - Control plane: manager
  - Data plane: Employee
  - Pods: Tasks
  - Manager assign tasks == Employee executes == Manager monitors.

➢ The below is the zoomed in version of Control plane node and Data plane node



- Control Plane Node

  c-c-m (cloud controller provider)
  - It connects Kubernetes with the cloud provider (AWS, GCP, Azure)
  -

  api (Api Server)
  - Entry point of kubernetes.
  - Every command (kubectl, UI, scripts) comes here.
  - Validates requests
  - Talks to other components.

  etcd (Database)
  - It is the database of Kubernetes.
  - Stores entire cluster data like, pods info, nodes info, configs.

  sched (Scheduler)
  - Decides where pods to run.
  - Choose the best worker node based on CPU/Memory, Availability.
  - It decides, "where should run?"

c-m (Controller Manager)

- Maintains desired state.
- Pods crashes → recreate it
- Nodes down → reschedule pods
- You can think it as a *Auto Healing System*.
- It decides "what should exist?"

- Data Plane Node

kubelet

- It runs on every worker node.
- Talks to API server.
- Ensures pods are running.
- What it does?

  Pods assigned to its node.

  Pull docker image, starts the container.

  Monitors health, restart if crashes.
- Scheduler assigns Pods, Kubelet runs that.

Kube Proxy

- Handles networking inside node.
- Routes traffic to correct pod.
- What it does?

  Maintains network rules (iptables/ipvs)

  Enable service → Pod communication.

Pods (Workloads)

- These are the actual applications.
- Each pod → 1 running instance.
- Inside each pod: container (your app), shared network, optional storage.

➢ Now the question is, **kubelet** restarts the containers (not pods) if crashes, then why CM exists.

- Lets say one node goes down, it means the kubelet present inside that node is also gone.
- Now CM checks if the required number of pods are running or not (according to the desired state)
- If not, then it creates new pods objects (not containers)
- After that, scheduler decides on which node that pods will be running.

- Now, **kubelet** present inside each node (that is decided), will run the containers inside the specific pods.
- One pods can run multiple containers, but they will share same ip and resources, it is advanced if you want to use logging or some functionality like that.
- **Kubelet restart the containers if crashes, because pods are just objects which is created by CM inside which Containers are run.**
-
  - F
  - F
  - F
  -

CM and Schd

- **CM**: We need 4 pods (create pods objects)
- **SCHD**: Where should these pods go? Pod-1 → Node-1, Pod-2 → Node-2 ..etc
- **Kubelet**: Runs on each worker node.
  It sees, Pod-1 is assigned to my node, then pull docker image, starts container and keep running.

Nodes vs Pods

- Pod is application level, it is what runs the container.
- Node, you can think of a computer, that can run multiple pods inside it.
- Now the question is, if a Node can run multiple Pods, then why do we need multiple Worker Nodes?
  - First thing is high availability and avoiding single point of failure.
  - 2nd reason can be cpu and ram.
    Lets say you have one node having 2 cores and 4 GB of ram.
    You have to run 4 pods, each will use 1 core and 2 GB of ram.
    Here, you need 2 worker node to run 4 pods.
    Pod-1, Pod-2 → Node-1, Pod-3, Pod-4 → Node-2
- You define how many app instances you want, and Kubernetes **automatically** creates pods and distributes them across nodes based on available resources and load balancing.
- F

Node, Kubernetes, VMs (provided by Cloud providers or Bare metal)

- You must be thinking what is the work of **c-c-m** and if all the nodes and things are there inside Kubernetes cluster, then why do we need cloud provider's VMs.
- First, What is a **Node**?
  - Its just a machine that runs multiple processes (services).
  - Its not necessary that the control plane processes and data plane processes will run in different Node, they can run in a single Node as well.
  - Lets say you install all inside one VM.
    - It'll have control plane processes like: **c-c-m** (only if VM is provided by a cloud provider), **c-m**, **etcd**, **sched**, **api**.
    - Also, it'll have data plane processes like: **pods**, **kubelet**, **k-proxy**.
    - Now, when you tell it the number of instances you want to create, same like before, **c-m** will create pods objects, **sched** will assign the pods to the node.

      As we have only node inside which all processes are running, so **sched** will assign this node for all the pods.

      So, all the pods will be running inside the same VM.
- Now, as we saw before, **c-m** checks if pods are gone (when node gets down) then it creates new pods objects to get the desired state and then **sched** will assign those pods to some nodes. So "Are the Nodes get created and deleted automatically?"
  - No, nodes are constant, only the pods are created.
  - We create VMs from a cloud provider and assign those to our Kubernetes cluster.
  - Now, the **sched** will schedule the *pods* to the available nodes only if it is up.
- Now, the question may be, if the nodes are directly assigned to the cluster, then what is the use of **c-c-m** to talk to cloud provider?
  - One of the use-case can be creating Load Balancer (ex: AWS ELB)
  - The flow is like the below:
    - User will trigger request to the Load Balancer (provided by Cloud Provider, Ex: AWS ELB).
    - Load Balancer doesn't know about the pods inside which the containers running our application are present.

      It only knows about the nodes. Because the nodes are VMs of the same Cloud Provider only. It can forward the request to those.
    - Now, lets say the Load Balancer sends the request to one node (Worker Node of course).

- Now, **k-proxy** is there inside the worker node which will decide to which pods the request should go to.

  Means, there can be multiple pods running inside the same Node, here **k-proxy** does its work.
- **User → Cloud Provider's Load Balancer → Worker Node (VM) → Pods**
- The user's request doesn't go to control plane. Because Control Plane is to manage the data plane, not for user's traffic.

- Other use cases of **c-c-m** are like
  - Storage (volumes)
  - Node lifecycle and metadata
  - Cloud networking (like pods from different nodes can communicate with each other)
- In simple words, **CCM** lets Kubernetes use cloud resources like storage, networking, and node info — not just load balancers.

- F
- F
- F
- F
- F
- F
- F
- F
- F
- F
- F
-